DATABASE REPORT

This report describes all the changes that we made to the skeleton code to complete Lab 1 of this database lab.

Design Decision

1. TupleDesc and Tuple

TupleDesc will store its schema as a private TDIItem[] which both constructors will build the array in a single loop where the unnamed constructor passes null field names. Later, the function equals() will compare the field types only (not the names). A hashmap sets their hashmap keys with finding their hashcode first and then checks for equality, so if two objects are equal then it must have the same hashcode.

2. Catalog

We use 4 hashmaps to back the catalog. The 4 hashmaps that we use are nameToId, idToFile, idToTupleDesc, and idToPkeyField which allows us to look up common id in O(1) or instant lookup time. First the function addTable() will check for an existing table with the same name and removes its stale entries from all three id-keyed maps before inserting. This process is going to satisfy the "replace on duplicate name" requirement. The only non constant operation is the getTableName(id), which scans nameToId for a matching id.

3. BufferPool

The page cache is made using ConcurrentHashMaps<PadeId, Page> plus an int capacity. getPage() will return a checked page ona hit and it will read the page from disk through Database.getCatalog().getDatabaseFile(...).readpage(...) on a miss. This evicts if the pool is at the capacity then it will cache and return it. We chose ConcurrentHashMap because we want the structure to be safe to share once locking is added in Lab3.

4. HeapFile and HeapFileIterator

The HeapFile constructor opens a single RandomAccessFile in "rw" mode and keeps it as a file. This acts as a helper so readPage() can seek any offset without reopening the file. getId() will return f.getAbsoluteFile().hashCode() which is a stable id that is unique with no global counter. Rather than an anonymous inner class, what we did was implement the iterator as a separate public class that keeps the page crossing logic readable.

5. SeqScan

SeqScan.getTupleDesc() does not return the table schema directly but it rebuilds a new TupleDesc. Where the field names are prefixed with the table alias so the column can stay distinguishable once tables are added in later labs. All iterations are delegated to the DbFileIterator obtained from the catalog.

CODE EXPLANATION

1. getNumTuples() at HeapPage.java Code:

```
111. return(int) Math.floor((BufferPool.getPageSize()*8)/(td.getSize() * 8 + 1));
```

The formula works in bits. The numerator, PageSize * 8, converts the page size into total bits available. The denominator, tupleSize * 8 + 1, is the cost per tuple in bits plus one header bit to track whether the slot is occupied. Dividing the two gives how many tuples fit, Math.floor() makes the rounding-down explicit, and (int) converts the result to an integer.

2. HeapPage.isSlotUsed() at HeapPage.java Code:

```
345. int byteIndex = i / 8;
346. int bitIndex = i % 8;
347. return (header[byteIndex] & (1 << bitIndex)) != 0;
```

For this function, we need to know first that the header is a byte array where each bit tracks whether a tuple slot is occupied. First, we **divide i by 8** because we want to know which byte in the array is our slot. Then we use **i modulo 8** to check the specific bit of our slot in the bytes. Then check if that specific slot is occupied, return true; otherwise, return false.

3. HeapFileIterator.hasNext() at HeapFileIterator.java Code:

```
74.while (++currentPageIndex < heapFile.numPages()){
75.  HeapPageId pid = new HeapPageId(heapFile.getId(), currentPageIndex); 76.
HeapPage page = (HeapPage) Database.getBufferPool().getPage(tid,
pid, Permissions.READ_ONLY);
77.    currentTupleIterator = page.iterator();
78.    if (currentTupleIterator.hasNext()) return true;}return false;
```

In the code, we use two cursors for the current page and the current tuple within that page. When a page runs out of tuples, we continue skipping empty until we find a tuple or run out of pages.

API MODIFICATION

We didn't make any changes to the provided API, but we did add a new helper class. HeapFileIterator implements the provided DbFileIterator interface to help iterate over tuples across multiple HeapPages. This helps improve code readability by separating the iteration logic from HeapFile while preserving the original given API. The logic for the new class can be found in CODE EXPLANATION (3).

Missing or Incomplete Elements

These skeleton methods remain intentional subs for later labs and return safe defaults, so Lab 1 tests still pass BufferPool.evictPage(), insertTuple(), deleteTuple(), flushPage(), flushAllPages(), discardPage(): Lab 2 (writes and eviction). HeapFile.writePage(), insertTuple(), deleteTuple(): Lab 2 (heap file mutation). BufferPool.holdsLock(), unsafeReleasePage(), transactionComplete(): Lab 3 (locking and transactions).

Two cleanup items remain in otherwise working code, which are SeqScan.getTupleDesc() contains a leftover System.out.println(names[i]) debug line that should be removed. HeapFile opens a RandomAccessFile but never closes it. Acceptable for short-lived test runs, but a resource leak in a long-running system.